POLITECH — Combined Application Documentation

Overview

POLITECH is a single-page, client-side police command & analytics web application composed of two HTML files that link to each other.

File	Title	Role
[police.html](file:///c:/Users/revan/OneDrive/Documents/politech/police.html)	Command Operations Center	Live operations hub — map, incidents, officer management
[analytics.html](file:///c:/Users/revan/OneDrive/Documents/politech/analytics.html)	Analytics Suite	Historical KPIs, charts, heatmap, export

Shared Design System

Both files share the same design language:

- **Font**: `Inter` (Google Fonts), weights 300–900
- **Icons**: Font Awesome 6.5.0
- **Color Tokens** (CSS custom properties):

Variable	Dark Value	Purpose
`--navy`	`#0f172a`	Page background
`--navy-2`	`#1e293b`	Panel/card background
`--navy-3`	`#334155`	Borders, muted fills
`--electric`	`#3b82f6`	Primary accent (blue)
`--emerald`	`#10b981`	Available / positive
`--crimson`	`#ef4444`	Critical / danger
`--amber`	`#f59e0b`	Warning / high priority
`--violet`	`#8b5cf6`	Supplementary accent
`--text`	`#e2e8f0`	Body text
`--text-muted`	`#94a3b8`	Secondary text
`--glass-bg`	`rgba(30,41,59,0.6)`	Glassmorphism card fill
`--glass-border`	`rgba(255,255,255,0.08)`	Glassmorphism card border

- **Light mode** is toggled via `[data-theme="light"]` on `<html>` and overrides the above tokens.
- **Transition**: `all 0.3s cubic-bezier(0.4,0,0.2,1)` on interactive elements.
- **Border radius**: `14px` (`--radius`).

`police.html` — Command Operations Center

Architecture

```
<body>
  Toast Container
  Command Center Drawer (left slide-over)
  Action Panel / Action Hub (right slide-over)
  Edit Assignment Modal
  <nav class="topbar">
  <div class="app-shell">
    <aside class="sidebar">
    <div class="main-content">
      Stats Bar
      page-dashboard  → Map + Incident Feed
      page-force      → Force Grid + Incident Drop Zones + Duty Roster
```

Navigation

- **Top bar** (`class="topbar"`): Logo, nav links, icon buttons (Command Center, New Assignment), theme toggle.
- **Sidebar** (`class="sidebar"`): Nav items for Live Map, Force Status, Quick Deploy, Analytics, Command Center. Shows a "Live Stats" mini-widget at the bottom.
- **`switchPage(page, el)`**: Hides all `.page-view` elements, shows `#page-{page}`, updates active states.

Stats Bar

Four tiles at the top of `main-content` showing live counts:

Element ID	Metric
`#st-active`	Active Duties
`#st-avail`	Officers Available
`#st-assigned`	Deployed Officers
`#st-done`	Completed Today

Dashboard Page (`#page-dashboard`)

Split layout: **1.2fr map** / **1fr incident feed**.

Live Tactical Map
- `<div id="googleMap">` — rendered by Google Maps JavaScript API (`v=weekly`).
- `initMap()` callback initialises the map at Chicago (`41.8781, -87.6298`), zoom 13, with a custom dark style (`DARK_MAP_STYLE`).
- Officers and incidents are plotted with custom inline SVG markers via `makeOfficerIcon(status)` and `makeIncidentIcon(priority)`.
- Clicking a marker opens a styled `InfoWindow`.
- Controls: `mapZoom(dir)` adjusts zoom ±1.
- **Custom Google Maps CSS** strips default controls and styles info windows to match the dark theme.

Incident Feed
- `renderFeed()` — renders `incidents[]` as draggable drop-targets.
- `addRandomIncident()` — pushes a randomly generated incident and re-renders.
- Each card supports drag-over/drop to quickly assign an officer.

Force Status Page (`#page-force`)

Force Grid
- `renderForceGrid()` renders `officers[]` as draggable cards with avatar initial, name, rank, badge, status pulse dot, and last-seen time.
- `panToOfficer(officerId)` switches to the Dashboard page and pans/zooms the map to the officer, then triggers their marker's click event after 450 ms.

Incident Drop Zones
- `renderIncidentDropZones()` shows unassigned incidents as drop targets on the Force page.
- `dropOnIncident(e, incId)` — validates the dragged officer is available, assigns them to the incident, creates a duty record, and calls `loadUI()`.

Active Duty Roster
- `renderRoster()` — lists non-completed duties with Complete ✓ / Edit ✎ / Delete 🗑 actions.

Action Hub (Slide-Over Panel)

Right-side drawer (`#actionPanel`, `width: 380px`). Form fields:

Field	ID	Type
Duty Type	`#dutyType`	`<select>`
Assign Officer	`#officer`	`<select>` (available only)
Location / Sector	`#location`	`<input text>`
Priority	`#priority`	`<select>`
Operational Notes	`#details`	`<textarea>`

On submit, a duty is pushed to `duties[]`, officer status set to `assigned`, and a success toast fires.

Command Center Drawer

Left-side drawer (`#alertDrawer`, `width: 360px`). `renderAlerts()` renders 5 static `alerts[]` items with icon, title, description, and timestamp. Types: `critical`, `warning`, `info`.

Edit Assignment Modal

`#editModal` — a centred modal with fields for Duty Type, Location, and Priority. On save, the duty record is updated in place.

Data Models

```js
// Officer
{ id, name, rank, badge, status: 'available'|'assigned'|'offduty', lastSeen, mapPos: {lat, lng} }

// Duty
{ id, type, officerId, officerName, location, details, priority, time, completed }

// Incident
{ id, type, location, priority: 'Low'|'Medium'|'High'|'Critical', time, assignedTo, mapPos: {lat, lng} }

// Alert
{ type: 'critical'|'warning'|'info', icon, title, desc, time }
```

Key JavaScript Functions

Function	Description
`initMap()`	Google Maps API callback — creates map and calls `renderMap()`
`renderMap()`	Clears & redraws all officer and incident markers
`makeOfficerIcon(status)`	Returns SVG data-URI marker icon for officers
`makeIncidentIcon(priority)`	Returns SVG data-URI marker icon for incidents
`panToOfficer(id)`	Pans map to officer's location, opens info window
`loadUI()`	Master refresh — calls all render functions + `updateStats()`
`updateStats()`	Syncs stat bar and sidebar live-stats widget
`renderFeed()`	Renders incident feed cards
`renderForceGrid()`	Renders officer drag-cards
`renderIncidentDropZones()`	Renders unassigned incidents as drop targets
`renderRoster()`	Renders active duty roster
`renderDropdown()`	Populates officer `<select>` with available officers
`dragStart(id)`	Sets `draggedOfficerId`
`dropOnIncident(e, incId)`	Handles officer → incident drag-and-drop assignment
`addRandomIncident()`	Generates and prepends a random incident
`toggleTheme()`	Switches dark / light data attribute
`toast(msg, type)`	Shows auto-dismissing toast notification
`openDrawer()` / `closeDrawer()`	Command Center drawer
`openAction()` / `closeAction()`	Action Hub panel
`completeDuty(id)`	Marks duty done, returns officer to available
`deleteDuty(id)`	Removes duty, frees officer
`editDuty(id)`	Opens edit modal pre-populated

Responsive Behaviour

```css
@media(max-width: 768px) {
  .sidebar { display: none; }
  .map-feed-layout { grid-template-columns: 1fr; }
  .feed-pane { display: none; }
  .analytics-grid { grid-template-columns: 1fr; }
```

```
}
```

`analytics.html` — Analytics Suite

Architecture

```
<body>
  Toast Container
  <nav class="topbar">
  <div class="page-wrap">
    Page Header (title + export buttons + date badge)
    KPI Row (4 cards)
    Row 1: Crime Density Heatmap | Response Time Gauge
    Row 2: Three Bar Charts (Incidents by Day, by Type, Officers Deployed)
    Row 3: Donut Chart (Priority Distribution) | Officer Performance Table
```

Page Header

- **Title**: "Analytics Suite" with chart-line icon.
- **Date badge**: displays `Mar 7, 2026`.
- **Export buttons**: "Export PDF" → `exportPDF()` (toast), "Export Excel" → `exportExcel()` (toast).

KPI Row

Four glassmorphism cards (`class="kpi-card"`) with a decorative blurred background circle:

Card	ID	Value	Trend
Total Incidents (MTD)	`#kpi-incidents`	**247**	↓ 12% from last month
Avg Response Time	—	**6.4 min**	↑ 8% faster than target
Critical Incidents Today	—	**18**	↓ 3 from yesterday
Duty Completion Rate	—	**94%**	↑ +2% this week

Crime Density Heatmap

- `buildHeatmap()` — IIFE that generates an **8 × 12** grid (8 sectors × 12 months).
- Each cell is assigned a random intensity value (0–1) and coloured:
  - **< 0.33** → blue (low)
  - **0.33–0.66** → amber (medium)
  - **> 0.66** → red (high)
- Row labels: `N, NE, E, SE, S, SW, W, NW`
- Column labels: `Jan` – `Dec`
- Hover: cell scales up to 1.25× with a coloured glow.

Response Time Gauge

- SVG semi-circle with a `linearGradient` from emerald → amber → crimson.
- Static `stroke-dashoffset: 90` represents 6.4 min.
- Below the gauge: Best / Avg / Worst tiles (`3.1`, `6.4`, `14.2` min).
- **By Shift table**:

Shift	Avg Time	Load
Morning (06–14)	5.2 min	Normal
Afternoon (14–22)	7.8 min	High
Night (22–06)	4.9 min	Low

Bar Charts

`buildBarChart(containerId, labelId, data, colors)` — renders proportional CSS-height bars with

tooltip on hover (`data-val` attribute + `::after` pseudo-element).

Chart	Data Points	Colors
Incidents by Day	Mon–Sun (32–61)	blue/amber
Incidents by Type	Patrol, Invest, Traffic, Desk, Emer	blue, violet, amber, green, red
Officers Deployed/Day	Mon–Sun (11–22)	green

> **Note**: `buildBarChart` is declared twice in the file (lines 396 and 422). The second declaration (with the fixed `(d,i)` index parameter) overrides the first.

Donut Chart — Priority Distribution

Inline SVG with four stacked `<circle>` arcs using `stroke-dasharray/offset` and `rotate` transforms:

Priority	Color	Share
Critical	`#ef4444`	25%
High	`#f59e0b`	20%
Medium	`#3b82f6`	30%
Low	`#10b981`	25%

Centre text shows **247 TOTAL**.

Officer Performance Table

Top-5 officers ranked by response performance:

#	Officer	Duties	Avg Time	Rating
1	Sarah Johnson	34	4.1 min	⭐ 98%
2	Lisa Rodriguez	29	5.2 min	⭐ 95%
3	James Wilson	31	5.8 min	⭐ 91%
4	Michael Chen	27	6.9 min	⭐ 87%
5	Robert Davis	22	8.1 min	⭐ 79%

Key JavaScript Functions

Function	Description
`toggleTheme()`	Switches dark / light mode
`toast(msg, type)`	Auto-dismissing notification toast
`exportPDF()`	Shows "Generating PDF report…" info toast
`exportExcel()`	Shows "Exporting Excel spreadsheet…" info toast
`buildHeatmap()` (IIFE)	Generates random crime density heatmap grid
`buildBarChart(cId, lId, data, colors)`	Renders proportional bar chart

Responsive Behaviour

```css
@media(max-width: 900px) {
  .kpi-row { grid-template-columns: repeat(2, 1fr); }
  .analytics-grid, .analytics-grid.three { grid-template-columns: 1fr; }
  .a-card.wide { grid-column: 1; }
}
@media(max-width: 600px) {
  .kpi-row { grid-template-columns: 1fr 1fr; }
  .donut-wrap { flex-direction: column; }
}
```

Inter-Page Navigation

From	Link	To
`police.html` topbar	`<a href="analytics.html">Analytics</a>`	`analytics.html`
`police.html` sidebar	`<a href="analytics.html">Analytics</a>`	`analytics.html`
`analytics.html` topbar logo	`<a href="police.html">`	`police.html`
`analytics.html` topbar nav	Dashboard / Force Status links → `police.html`	`police.html`

Known Issues / Notes

> [!NOTE]
> `buildBarChart` is declared twice in `analytics.html`. The second definition (line 422) is the correct one — it fixes the bar index lookup from `data.indexOf(d)` to the proper `(d, i)` parameter. This causes no runtime error since JS hoists and the later `const` assignment wins in a linear script, but the duplicated dead code can be cleaned up.

> [!NOTE]
> The Google Maps API is loaded without an API key (`v=weekly` dev mode). For production, replace with a valid key to avoid quota/watermark issues.

> [!NOTE]
> All analytics data (KPIs, heatmap intensities, chart values) is hardcoded or randomly generated client-side. There is no backend or persistence layer.